

Plano Interactivo 2D

Android · Kotlin · Jetpack Compose — Base de conocimiento técnico antes del desarrollo

Kotlin

Jetpack Compose

Retrofit

Compose Canvas

BOM 2025.12.00

STACK APP	BACKEND	PERSISTENCIA
Kotlin + Compose 1.10 (BOM 2025.12.00) · Retrofit	Endpoint REST .NET (existente, fuera de alcance)	Por decidir (Room como caché opcional)

Objetivo: generar la base de conocimiento necesaria para implementar un plano 2D editable e interactivo dentro de la app, antes de comenzar el desarrollo.

1. PREGUNTAS DE INVESTIGACIÓN — RESPUESTAS

1.1. Opciones para renderizar planos interactivos 2D en Jetpack Compose

Hay cuatro caminos viables. La elección depende de si el plano se **dibuja procedimentalmente desde datos** (caso de este proyecto) o se **carga desde un archivo SVG** producido por otra herramienta.

Opción	Cuándo usarla	Notas clave
Compose Canvas / drawBehind / drawWithCache	El plano se genera a partir de datos (zonas en una BD). Caso de este proyecto.	API nativa Compose (Skia). Soporta drawRect, drawPath, transformaciones, gradientes. Opción de primer nivel recomendada por Google. ■ Elegida
Coil 3 + coil-svg	El plano se entrega como SVG (estático o remoto) con zonas interactivas encima.	Coil 3 estable en todas las plataformas; Android respaldado por AndroidSVG. Alternativa viable.
AndroidView + SVGImageView (AndroidSVG)	SVGs muy complejos cargados en runtime donde Coil falla.	Overhead significativo. Última opción.
ImageVector / VectorPainter	Asset fijo conocido en compile-time.	No sirve si el plano viene de la BD o del endpoint.

Recomendación: Compose Canvas puro. El plano se construye desde una lista de Zone provenientes del endpoint .NET; no hay SVG que cargar. Las otras opciones son alternativas válidas solo si el backend empieza a entregar planos pre-renderizados como SVG.

1.2. Sistema de coordenadas en Compose Canvas y mapeo de zonas

Compose Canvas usa origen (0,0) en la esquina superior izquierda, X creciendo a la derecha, Y hacia abajo. Las unidades son **pixels** dentro del scope de dibujo.

Tres espacios a separar:

- **Espacio del modelo (world coordinates):** unidades lógicas del plano independientes del dispositivo. Ejemplo: Zone(x=100, y=200, width=50, height=40).
- **Espacio del lienzo (canvas coordinates):** pixels reales dentro del Canvas composible, dependientes del tamaño del contenedor.
- **Espacio de pantalla con transformación:** después de aplicar pan y zoom con graphicsLayer.

Patrón recomendado:

- Guardar zonas en unidades del modelo (no pixels).
- Calcular factor de escala $\text{worldToCanvas} = \text{canvasSize} / \text{worldSize}$ manteniendo aspect ratio.
- Aplicar scale y translate del usuario por encima, usando graphicsLayer.
- Para hit-testing de gestos, aplicar la transformación inversa al punto del toque.

```
fun screenToWorld(  
    point: Offset,  
    pan: Offset,  
    scale: Float,  
    worldToCanvas: Float  
): Offset = (point - pan) / scale / worldToCanvas
```

1.3. Madurez y eficiencia: Compose Canvas vs. Canvas tradicional de Views

Compose Canvas es un wrapper sobre android.graphics.Canvas (Skia por debajo), por lo que en rendimiento crudo son equivalentes para dibujo.

■ Integración natural con State, recomposición selectiva ■
drawWithCache para evitar recomputaciones costosas ■
Composabilidad nativa con el resto de la jerarquía ■
Benchmarks de scroll igualan Views (dic. 2025) ■ SVG
complejo cargado en runtime requiere librería externa

■ Si ya existe una View custom muy optimizada con
invalidate() selectivo ■ API legacy muy estable y
documentada ■ Integración forzada con Compose
(AndroidView) ■ Estado no reactivo por defecto ■ Para
proyectos nuevos: overhead sin beneficio

Optimización clave: usar Modifier.drawWithCache { onDrawBehind { ... } } para memoizar Path, Brush y otros objetos costosos entre recomposiciones. Crítico cuando hay muchas zonas.

1.4. Manejo de gestos táctiles (tap, drag, pinch-to-zoom)

Compose ofrece tres niveles de abstracción. La regla es preferir modificadores de alto nivel sobre manejo custom, porque agregan más funcionalidad encima de los eventos crudos.

Nivel	API	Cuándo usarlo
Nivel 1 — Alto nivel	Modifier.clickable, Modifier.draggable, Modifier.transformable	Taps, drag unidireccional, pan+zoom+rotate. Preferir siempre que alcancen.
Nivel 2 — Detectores en pointerInput	detectTapGestures, detectDragGestures, detectTransformGestures	Control granular del ciclo de vida del gesto. Caso de este proyecto.
Nivel 3 — Eventos crudos	awaitPointerEventScope	Gestos custom no cubiertos (ej. drag-after-long-press + multi-touch simultáneo).

Patrón recomendado para este proyecto:

```
@Composable
fun PlanoInteractivo(zones: List<Zone>, onZoneMoved: (Zone) -> Unit) {
    var scale by remember { mutableFloatStateOf(1f) }
    var offset by remember { mutableStateOf(Offset.Zero) }
    Box(
        modifier = Modifier
            .fillMaxSize()
            // Capa 1: pan + zoom global del lienzo
            .pointerInput(Unit) {
                detectTransformGestures { _, pan, zoom, _ ->
                    scale = (scale * zoom).coerceIn(0.5f, 5f)
                    offset += pan
                }
            }
        .graphicsLayer(
            scaleX = scale, scaleY = scale,
            translationX = offset.x, translationY = offset.y
        )
    ) {
        Canvas(modifier = Modifier.fillMaxSize()) {
            zones.forEach { zone ->
                drawRect(color = zone.color,
                    topLeft = Offset(zone.x, zone.y),
                    size = Size(zone.width, zone.height))
            }
        }
        // Capa 2: overlay con drag individual por zona
        zones.forEach { zone -> ZoneDragOverlay(zone, scale, onZoneMoved) }
    }
}
```

Conflictos típicos y resolución:

Conflicto	Solución
Drag de zona vs. pan del lienzo	Capas separadas de pointerInput. La zona consume el evento con change.consume().

Tap vs. doble-tap	Si se provee onDoubleTap, el sistema espera brevemente antes de confirmar el tap simple.
Pinch en plano vs. scroll del padre	Box con Modifier.pointerInput que consume gestos multi-touch antes de que lleguen al LazyColumn padre.

1.5. Consumo del endpoint .NET y persistencia local

Hay dos arquitecturas según los requisitos del producto.

Contrato del endpoint .NET:

```
GET /spaces/{id}/zones → List<ZoneDto>
POST /spaces/{id}/zones → ZoneDto (zona creada con id)
PUT /spaces/{id}/zones/{zoneId} → ZoneDto (zona actualizada)
DELETE /spaces/{id}/zones/{zoneId} → 204 No Content
```

DTO esperado (JSON):

```
{ "id": 123, "spaceId": 45, "name": "Zona A",
  "x": 100.0, "y": 200.0, "width": 50.0, "height": 40.0,
  "color": "#FF5733", "rotation": 0.0 }
```

Opción A — Solo API (sin caché local)

```
class ZoneRepository(private val api: ZonesApi) {
    suspend fun getZones(spaceId: Long): List<Zone> =
        api.getZones(spaceId).map { it.toDomain() }
    suspend fun createZone(spaceId: Long, zone: Zone): Zone =
        api.createZone(spaceId, zone.toDto()).toDomain()
}
```

Opción B — API + Room (offline-first):

```
@Entity(tableName = "zones")
data class ZoneEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val serverId: Long?, // null hasta sincronizar
    val spaceId: Long,
    val name: String,
    val x: Float, val y: Float,
    val width: Float, val height: Float,
    val colorArgb: Int,
    val rotation: Float = 0f,
    val syncStatus: SyncStatus // PENDING, SYNCED, ERROR
)
```

Tabla de decisión:

Pregunta	Si la respuesta es...	Indica...
¿La app debe funcionar sin conexión?	Sí	Opción B

¿Se acepta un loading visible al abrir?	No	Opción B
¿Varios usuarios editan el mismo plano?	Sí	Opción B + resolución de conflictos
¿Zonas cambian muy frecuentemente?	Sí	Opción A + tiempo real (SignalR)
¿Hay tiempo para implementar la sync?	No	Opción A primero

Recomendación técnica del spike: empezar por la Opción A para el MVP. Permite tener un funcionamiento rápido. Agregar Room encima del repositorio es una refactorización acotada, no una reescritura.

1.6. Plantillas open-source y ejemplos SVG

- GitHub topic `jetpack-compose-canvas` — incluye apps de drag-and-drop de cajas a áreas específicas, muy cercano al caso de este spike.
- Gist `ardakazanci "Interactive Canvas for jetpack compose"` — punto de partida directo para un lienzo zoomable/paneable.
- SVGs genéricos (Floorplanner, RoomSketcher, Figma) — solo aplican si se elige la opción Coil + SVG.

Para este proyecto, lo más rentable es construir el Canvas desde cero (~150 líneas de código). Se entiende mejor el comportamiento y no hay dependencia de formato externo.

1.7. Edición dinámica (agregar, mover, redimensionar, eliminar)

La arquitectura reactiva de Compose hace esto trivial independientemente de si los datos vienen solo del API o de Room.

- `Flow<List<Zone>>` o `StateFlow` desde el Repository → cualquier mutación se refleja automáticamente en la UI.
- Estado intermedio en `ViewModel` para zonas en edición (no persistidas), escribir al API/BD solo en `onDragEnd`.
- Modos de edición: `enum EditMode { VIEW, ADD, MOVE, RESIZE, DELETE }` controlado por el `ViewModel`.

```
class PlanoViewModel(private val repository: ZoneRepository) : ViewModel() {
    val mode = MutableStateFlow(EditMode.VIEW)
    val zones: StateFlow<List<Zone>> = /* desde repository */
    fun onZoneDragEnd(zone: Zone) = viewModelScope.launch {
        repository.updateZone(spaceId, zone)
    }
    fun addZone(rect: Rect) = viewModelScope.launch {
        repository.createZone(spaceId, Zone(id = 0, rect = rect, ...))
    }
}
```

- Para redimensionar: dibujar handles (cuadrados pequeños) en las esquinas de la zona seleccionada y aplicar `detectDragGestures` sobre cada handle modificando x/y o width/height según su posición.

1.8. Límites de rendimiento con muchas zonas

Punto crítico	Mitigación
Número de zonas	Dibujar todo en un único Canvas, no un composabile por zona.
Recomposición excesiva	graphicsLayer con lambdas en vez de pasar valores directamente — evita la fase de composición.
Viewport culling	Para 10.000+ zonas: dibujar solo las que intersectan el viewport visible.
Bitmaps cacheados	Fondos estáticos costosos: drawWithCache + ImageBitmap precomputado.
Path complejos	Construir Path una sola vez (cachear en Zone o con remember) y reutilizar.

Regla práctica: para los casos típicos de este proyecto (decenas a bajos cientos de zonas) ningún truco especial es necesario. Las optimizaciones entran a partir de ~1000 zonas o cuando el dispositivo objetivo es de gama baja.

1.9. Responsividad en diferentes pantallas

Tamaño físico del lienzo: BoxWithConstraints para conocer maxWidth/maxHeight y calcular worldToCanvas = min(maxWidth/worldWidth, maxHeight/worldHeight).

Densidad de pixels: Guardar todo en world units y convertir solo al dibujar. Para grosor de línea invariante al zoom, dividir strokeWidth por scale.

Orientación: Recalcular worldToCanvas en cada cambio. Como BoxWithConstraints reacciona automáticamente, esto es gratuito.

1.10. Compose Canvas vs. enfoque híbrido con Android Views

Criterio	Compose Canvas	Híbrido (AndroidView)
Una sola fuente de verdad para la UI	■	■
Estado reactivo nativo	■	■
Sin overhead de interop	■	■
Recomposición selectiva	■	■
SVGs complejos cargados en runtime	■	■
Reutilizar View custom ya optimizada	■	■
Gestos sin competencia entre capas	■	■

Recomendación: Compose Canvas puro. El enfoque híbrido solo se justifica si aparece un requisito específico que Canvas no resuelve bien (SVG complejo en runtime, o reutilizar una View muy optimizada ya existente en el proyecto).

2. TABLA COMPARATIVA FINAL

Criterio	Compose Canvas	Coil + SVG	AndroidView + AndroidSVG	Canvas tradicional
Rendimiento	Excelente (Skia nativo)	Bueno para mostrar; limitado para editar	Aceptable, con overhead de interop	Excelente
Mantenimiento	API oficial activa	Activo (Coil 3 estable)	Estable pero menos activa	API legacy estable
Documentación	Oficial, extensa	Buena	Suficiente pero antigua	Muy extensa, no Compose-first
Integración Compose	Nativa	Buena (vía composables)	Forzada (AndroidView)	Forzada
Soporte gestos	Excelente (pointerInput)	Encima del AsyncImage	Mezcla complicada	Listeners de View
Edición dinámica	Trivial (Compose state)	Posible pero forzado	Difícil	Posible pero verboso
Consumo endpoint .NET	Trivial (datos puros)	Igual	Igual	Igual
Persistencia local (Room)	Trivial (datos puros)	Igual	Igual	Igual
Curva de aprendizaje	Media (gestos avanzados)	Baja	Media (interop)	Alta (XML + custom views)
Recomendación	■ ELEGIDA	Alternativa si hay SVG remoto	Solo último recurso	Descartada

3. CONCLUSIÓN Y DECISIÓN

Alternativa seleccionada para renderizado: Compose Canvas puro (con `drawBehind` o `drawWithCache` según el caso).

Stack seleccionado:

Capa	Librería / API	Propósito
UI	Jetpack Compose	Composables del plano y de la app
Renderizado del plano	Compose Canvas + <code>pointerInput</code>	Dibujar zonas y manejar gestos
Estado de UI	<code>ViewModel</code> + <code>StateFlow</code>	Coordinar lógica y exponer estado a la UI
Red	<code>Retrofit</code> + <code>OkHttp</code> + <code>Kotlinx Serialization</code>	Consumir el endpoint .NET
Inyección de dep.	<code>Hilt</code> (o manual)	Conectar las capas

Justificación del renderizado:

- El plano se construye desde datos (zonas en el endpoint .NET), no desde un asset SVG → descarta Coil/AndroidView.
- Compose Canvas es la opción nativa sin overhead de interop.
- La integración con gestos avanzados y el estado Compose es directa.
- El rendimiento es suficiente para los volúmenes esperados (decenas a cientos de zonas).

Riesgos identificados y mitigaciones:

Riesgo	Mitigación
Conflictos de gestos (drag vs. pan global)	Capas separadas de <code>pointerInput</code> , uso disciplinado de <code>change.consume()</code>
Recomposición costosa con muchas zonas	Un solo Canvas para todas las zonas + <code>drawWithCache</code> ; <code>graphicsLayer lambda</code> para transformaciones
Curva de aprendizaje en <code>pointerInput raw</code>	Comenzar con detectores de alto nivel y bajar a raw solo si es necesario
Latencia visible del endpoint (Opción A)	Migrar a Opción B (API + Room) si la UX lo requiere

Conflictos de sync si se elige Opción B

Política last-write-wins inicial; revisar si el producto requiere edición concurrente

4. PRÓXIMOS PASOS SUGERIDOS

1 Crear rama spike/plano-2d-prototipo

Construir prototipo mínimo: lienzo con 3 rectángulos arrastrables + pan/zoom global + consumo del endpoint .NET (Opción A).

2 Validar con diseño UX

¿El gesto de pan-mientras-se-arrastra-una-zona se siente natural? ¿La latencia del endpoint es aceptable?

3 Definir modos de edición del producto

¿Edición libre? ¿Solo arrastrar? ¿Con grid snap?

4 Decidir con producto sobre modo offline

Si sí, abrir issue para implementar la Opción B (API + Room).

5 Cerrar spike #297

Abrir issues de implementación.

5. REFERENCIAS

1. Jetpack Compose — Multi-touch gestures

developer.android.com/develop/ui/compose/touch-input/pointer-input/multi-touch

2. Jetpack Compose — Understand gestures

developer.android.com/develop/ui/compose/touch-input/pointer-input/understand-gestures

3. Guía de arquitectura Android — capa de datos

developer.android.com/topic/architecture/data-layer

4. Guía oficial — offline-first

developer.android.com/topic/architecture/data-layer/offline-first

5. Android Developers Blog — Compose December '25 release (BOM 2025.12.00)

android-developers.googleblog.com

6. Retrofit

square.github.io/retrofit

7. Room

developer.android.com/training/data-storage/room

8. Coil 3 changelog — soporte SVG

coil-kt.github.io/coil/changelog

9. GitHub topic: jetpack-compose-canvas

github.com/topics/jetpack-compose-canvas